Abstract

This report presents the development and evaluation of a compact neural network for the game of Go, designed under a strict constraint of fewer than 100,000 trainable parameters. The architecture combines convolutional layers with modern attention mechanisms to capture both local and global spatial features efficiently.

The network is trained to predict both move probabilities and the value of board positions, using data augmentation to enforce symmetry invariance and improve generalization. Experimental results demonstrate that the model achieves strong policy accuracy and reliable value estimation, highlighting the effectiveness of hybrid architectures under tight parameter budgets.

These findings provide insights into designing efficient and expressive models for strategic board games, where maximizing performance while minimizing model size is crucial.

Contents

Chapter 1

Introduction

The game of Go is widely regarded as a challenging benchmark for artificial intelligence due to its extremely large state space, long-term strategic dependencies, and the difficulty of accurately evaluating board positions. Unlike many other board games, Go offers very limited heuristics, making it particularly well-suited for learning-based approaches.

In recent years, major breakthroughs in Go-playing systems have been achieved by combining deep neural networks with self-play and Monte-Carlo Tree Search (MCTS) (Kocsis and Szepesvári (2006)). These systems rely on two key components: a *policy network*, which predicts strong candidate moves, and a *value network*, which estimates the probability of winning from a given board position. Inspired by these advances, the goal of this project is to train a compact deep neural network capable of jointly learning both policy and value predictions for the game of Go.

This project is based on an existing Go environment provided as part of the course. The dataset, generated from high-quality self-play games using the *KataGo* engine (Wu (2020)), as well as the game engine and data interface, were supplied by the instructor. As a result, the focus of this work is not on data generation or game simulation, but on the design, implementation, and training of neural network architectures under fixed experimental conditions.

The learning task consists in supervised training of a joint policy–value network. Given a Go board position represented by a set of predefined input feature planes, the network must predict a probability distribution over all possible moves (policy) and an estimate of the probability of winning for the current player (value). To ensure fairness and computational efficiency, a strict constraint is imposed: the total number of trainable parameters must remain below 100,000.

The main contribution of this project lies in an iterative experimental approach. Five different versions of neural network architectures were designed and implemented, each introducing architectural or training modifications compared to the previous one. These versions explore variations of convolutional and residual designs, attention mechanisms, lightweight channel-wise modulation, data augmentation through board symmetries, and different training strategies.

This report presents and compares the six implemented versions, with a focus on architectural choices and their impact on training behavior. Rather than aiming for state-of-the-art performance, the objective is to gain practical insight into how compact neural architectures for Go can be progressively refined within strict resource constraints.

Chapter 2

Data Handling and Augmentation

This part presents the data interface used to load training samples, the representation of input features and learning targets, and the data augmentation strategy applied during training.

2.1 Provided Data Interface

The dataset and the data-loading interface are provided as part of the project framework. Training and validation data are accessed through `golois` Python module, which exposes dedicated functions to retrieve batches of Go positions together with their associated policy and value targets. No manual preprocessing or dataset construction is required.

Each training sample corresponds to a board position extracted from self-play games generated by the *KataGo* engine. The data is returned directly in tensor form and can be used immediately for training the neural network.

2.2 Input Features and Output Targets

Each Go position is encoded as a tensor of shape $(19, 19, 31)$, corresponding to the standard Go board size and a stack of feature planes. These planes are provided by the dataset and are designed to capture both the current state of the game and short-term historical information.

The input representation includes, among others:

- the color of the player to move,
- the current board configuration encoded on separate planes,
- ladder-related features,
- several previous board states, allowing the network to infer temporal patterns such as captures or *ko* situations.

All input values are stored as floating-point numbers and are directly fed to the network without additional normalization, as the planes are already encoded in a consistent binary or bounded format.

The network is trained to predict two complementary outputs: a **policy target** and a **value target**.

The **policy target** is a vector of size 361, corresponding to the 361 intersections of the 19×19 Go board. For each training example, this vector represents the move selected during self-play. In the simplest case, it is encoded as a one-hot vector with value 1.0 for the played move and 0.0 elsewhere, and can be interpreted as a probability distribution over possible moves.

The **value target** is a scalar in the range $[0, 1]$ representing the estimated probability that the white player will win the game from the given position. This value is provided by the Monte-Carlo Tree Search used during self-play and reflects a strong evaluation of the position rather than the final game outcome.

Both targets are learned jointly during training, allowing the network to associate move selection with position evaluation.

2.3 Data Augmentation

To improve generalization and exploit the inherent symmetries of the Go board, data augmentation based on geometric transformations is applied during training. The Go board is invariant under the dihedral group D_4, which includes rotations by multiples of 90° and horizontal reflections.

For each training sample, a random symmetry is applied on-the-fly during training. This transformation is consistently applied to both:

- the input board tensor, by rotating and/or flipping the spatial dimensions,
- the policy target, by reshaping it into a 19×19 grid, applying the same transformation, and flattening it back to a vector.

The value target remains unchanged under these transformations, as it represents a global evaluation of the position. This augmentation strategy effectively increases the diversity of training data without increasing the dataset size or introducing artificial noise, while enforcing symmetry awareness as an inductive bias for the model.

Data augmentation is applied at each training iteration, ensuring low memory overhead and increased variability across epochs.

Chapter 3

Neural Network Architectures and Training

A sequence of neural network architectures is explored for predicting moves and evaluating positions in the game of Go. The models are progressively refined, starting from compact residual baselines and advancing to hybrid convolutional architectures enhanced with attention mechanisms, all while maintaining a strict limit on the number of trainable parameters.

3.1 Version 1: Compact Residual Policy–Value Network

This first version establishes a solid baseline architecture for the project. The objective is to design a compact policy–value neural network for the game of Go that balances expressive power and computational efficiency, while strictly respecting the constraint of fewer than 100,000 trainable parameters.

The architecture and training setup emphasize stability, regularization, and reproducibility, making this version a reliable reference for comparison with subsequent models.

3.1.1 Network Architecture

The network follows a shared-trunk policy–value design. Go board positions are provided as input tensors of shape $(19, 19, 31)$, as described in Chapter 2. All convolutional layers are implemented without bias terms and include L_2 weight regularization.

An initial 1×1 convolution projects the 31 input feature planes into a latent representation with 36 channels. This projection layer is followed by batch normalization and a ReLU activation, allowing efficient mixing of input features while keeping the parameter count low.

The shared trunk consists of four residual blocks. Each residual block contains two 3×3 convolutional layers with 36 filters, each followed by batch normalization and a ReLU activation. Identity skip connections are used to add the input of each block to its output before the final activation. These residual connections improve gradient flow and enable deeper representations without degradation in training performance.

From the shared representation, the network splits into two task-specific heads:

- The **policy head** applies a 1×1 convolution to reduce the channel dimension to one, followed by flattening and a softmax activation. The output is a probability distribution over the 361 legal board positions.

- The **value head** begins with a 1×1 convolution followed by batch normalization and a ReLU activation. Global average pooling is then used to aggregate spatial information. This pooled representation is passed through a fully connected layer with 50 hidden units and a dropout rate of 0.2, before a final sigmoid activation produces a scalar value in $[0, 1]$.

With this configuration, the complete model contains exactly **95,951 trainable parameters**, remaining comfortably below the imposed limit of 100,000 parameters.

3.1.2 Hyperparameters

The main architectural and training hyperparameters for this version are summarized below:

- Number of input feature planes: 31
- Board size: 19×19
- Number of convolutional filters: 36
- Number of residual blocks: 4
- Hidden units in value head: 50
- Dropout rate (value head): 0.2
- Batch size: 128
- Number of training epochs: 200

An L_2 regularization coefficient of 10^{-3} is applied to all trainable layers to limit overfitting.

3.1.3 Optimization and Loss Functions

The model is trained using the Adam optimizer with an initial learning rate of 10^{-3}. Gradient clipping with a maximum norm of 1.0 is applied to improve numerical stability throughout training.

The policy head is optimized using categorical cross-entropy with label smoothing set to 0.1, which reduces overconfidence and improves generalization. The value head is trained using mean squared error. The total loss is defined as a weighted sum of the two objectives, with a higher weight assigned to the policy loss in order to prioritize accurate move prediction.

3.1.4 Training Procedure and Evaluation

Training is performed in an online fashion using batches generated on-the-fly through the provided `golois` interface. Data augmentation via board symmetries is applied during training, while evaluation is performed on non-augmented validation data, as described in Chapter 2.

Early stopping based on the training loss is used to reduce overfitting, and the best-performing model is saved automatically using model checkpointing. During training, several metrics are monitored, including total loss, policy accuracy, policy loss, value loss, and value mean absolute error.

This first version provides a stable and well-regularized baseline, serving as a strong foundation for the exploration of more advanced architectures in subsequent versions.

3.2 Version 2: Simplified AlphaZero-Style Architecture

The second version explores a more streamlined AlphaZero-inspired architecture (Silver et al. (2018)), aiming to improve training stability and efficiency while remaining within the strict parameter budget. Compared to Version 1, this model reduces architectural complexity in the shared trunk and simplifies the value head, while slightly increasing the width of the convolutional layers.

This version focuses on evaluating whether a shallower network with wider feature maps and fewer regularization mechanisms can achieve comparable performance.

3.2.1 Network Architecture

The input representation remains unchanged, with Go board positions encoded as tensors of shape $(19, 19, 31)$. All convolutional layers are regularized using L_2 weight decay and are implemented without bias terms unless explicitly stated.

Feature extraction begins with a 3×3 convolution using 39 filters, followed by batch normalization and a ReLU activation. In contrast to Version 1, no initial 1×1 projection is used, allowing the network to immediately capture local spatial patterns.

The shared trunk consists of three residual blocks. Each block follows the standard residual structure with two 3×3 convolutions, batch normalization, and ReLU activations, connected through identity skip connections. The reduced depth of the trunk is compensated by a slightly larger number of filters per layer.

After the shared representation, the network branches into two output heads:

- The **policy head** applies a 1×1 convolution to produce a single-channel feature map, which is then flattened and normalized using a softmax activation to obtain a probability distribution over the 361 board positions.
- The **value head** directly applies global average pooling to the shared feature maps, followed by a single fully connected layer with 32 hidden units and ReLU activation. A final sigmoid unit outputs the estimated win probability for the current player.

This design results in a total of **95,460 trainable parameters**, remaining below the imposed limit while providing a slightly wider but shallower alternative to Version 1.

3.2.2 Hyperparameters

The principal hyperparameters used in this version are listed below:

- Number of input feature planes: 31
- Board size: 19×19
- Number of convolutional filters: 39
- Number of residual blocks: 3
- Hidden units in value head: 32
- Batch size: 128

- Number of training epochs: 500

A reduced L_2 regularization coefficient of 10^{-4} is used compared to Version 1, reflecting the simpler architecture and the absence of dropout.

3.2.3 Optimization and Loss Functions

Training is performed using the Adam optimizer with a learning rate of 10^{-3}. Unlike Version 1, no gradient clipping or label smoothing is applied, allowing for a more direct optimization of the objective functions.

The policy head is trained using categorical cross-entropy, while the value head uses mean squared error. The total loss is defined as a weighted sum of the two components, with relative weights of 1.0 for the policy loss and 0.4 for the value loss. Policy accuracy and value mean absolute error are monitored during training.

3.2.4 Training Procedure and Evaluation

As in the previous version, training is conducted in an online fashion using batches generated by the `golois` framework. Each training batch is augmented using a randomly sampled symmetry from the dihedral group of the Go board, while validation is always performed on non-augmented data, as described in Chapter 2.

The training loop runs for up to 500 epochs, with periodic evaluation on a fixed validation set. No early stopping is applied in this version, allowing the network to fully exploit the longer training schedule.

This second version provides a cleaner and more minimal baseline, enabling a direct comparison between deeper, more regularized architectures and shallower, wider designs under identical parameter constraints.

3.3 Version 3: Hybrid Convolutional and Attention-Based Architecture

The third version explores a more expressive neural network architecture by combining a deeper convolutional backbone with local multi-head attention mechanisms. The objective is to enhance the modeling of spatial dependencies on the Go board while maintaining a compact model size compatible with the imposed parameter budget.

This version represents a clear step forward in architectural complexity compared to the previous models, aiming to capture richer positional patterns and interactions between stones.

3.3.1 Network Architecture

The input to the network is a tensor of shape $(19, 19, 31)$ representing the current Go position. Feature extraction begins with a 3×3 separable convolution that projects the input planes into a representation with 61 channels. Separable convolutions are chosen to reduce the number of parameters while allowing the network width to increase.

The shared trunk consists of six residual blocks. Each block is built from two depthwise separable convolutions followed by batch normalization and ReLU activations. Residual

connections are used to preserve gradient flow and enable deeper architectures without performance degradation.

Following the convolutional trunk, two consecutive local multi-head attention blocks are applied. These blocks introduce an explicit mechanism for modeling interactions between neighboring board positions. Queries, keys, and values are generated using 1×1 convolutions, while keys and values are further processed by depthwise convolutions to enforce spatial locality. Attention is computed independently across four heads and reintegrated into the feature maps through residual connections.

The network then branches into two task-specific heads:

- The **policy head** applies a 1×1 convolution followed by a flattening operation and a softmax activation, producing a probability distribution over the 361 legal moves.
- The **value head** uses global average pooling to aggregate spatial information, followed by a fully connected layer with 192 hidden units and ReLU activation. A final sigmoid layer outputs a scalar value representing the estimated probability of winning.

Thanks to the use of separable convolutions and compact attention blocks, the total number of trainable parameters is kept at **98,767**.

3.3.2 Hyperparameters

The main hyperparameters used in this version are summarized below:

- Number of input feature planes: 31
- Board size: 19×19
- Number of convolutional filters: 61
- Number of residual blocks: 6
- Attention embedding dimension: 56
- Number of attention heads: 4
- Hidden units in the value head: 192
- Batch size: 128
- Number of training epochs: 1000

An L_2 regularization term with coefficient 10^{-4} is applied to convolutional and dense layers to mitigate overfitting.

3.3.3 Optimization and Loss Functions

The model is trained using the Adam optimizer with a learning rate of 10^{-3}. Training is performed in a multi-task setting, jointly optimizing the policy and value predictions.

The policy head is optimized using categorical cross-entropy loss, while the value head is trained using mean squared error. The total loss is defined as a weighted sum of these two components, with weights set to 1.0 for the policy loss and 0.4 for the value loss. This weighting reflects the primary importance of accurate move selection while still enforcing reliable position evaluation.

3.3.4 Training Procedure and Evaluation

Training follows an online data generation scheme in which new batches of positions are provided at each epoch. For every training batch, data augmentation based on random symmetries of the Go board is applied to both the input positions and the policy targets, while the value targets remain unchanged.

The model is trained for up to 1000 epochs with a mini-batch size of 128. Validation is performed periodically on a fixed validation set without data augmentation to ensure consistent evaluation. Performance is monitored through the total loss, policy accuracy, and mean absolute error of the value prediction.

This third version constitutes the most expressive architecture among the explored previous models, combining increased depth and attention-based mechanisms to achieve stronger empirical performance while remaining within the prescribed parameter constraints.

3.4 Version 4: Hybrid ConvNeXt and Attention-Enhanced Architecture

The fourth version introduces a more advanced hybrid architecture that combines modern convolutional design principles with multiple lightweight attention mechanisms. This model aims to further improve representational capacity while preserving training stability and remaining below the strict parameter budget.

Compared to previous versions, this architecture emphasizes richer channel-wise interactions and broader spatial context, while keeping the computational cost under control.

3.4.1 Network Architecture

The network takes as input a tensor of shape $(19, 19, 31)$ representing the Go board state. A convolutional stem composed of a 3×3 convolution, batch normalization, and ReLU activation is first applied to extract low-level spatial features and project the input into a 47-channel representation.

The core of the network consists of six stacked *hybrid blocks*. Each hybrid block is built around a ConvNeXt-inspired structure (Liu et al. (2022)) using a large 7×7 depthwise convolution followed by a pointwise bottleneck expansion with GELU activation. This design allows the network to capture broader spatial context while maintaining parameter efficiency.

To further enhance feature expressiveness, each ConvNeXt block is augmented with several lightweight attention mechanisms applied sequentially:

- **Local Channel Attention (LCA)**, which reweights feature channels based on global average statistics,
- **Squeeze-and-Excitation (SE)**, reinforcing informative channels through a compact gating mechanism,
- **Global Context (GC) block**, which injects global contextual information back into the feature maps.

Each hybrid block is wrapped in a residual connection, ensuring stable gradient propagation and enabling deeper representations.

After the shared trunk, the network branches into two heads:

- The **policy head** applies a 1×1 convolution followed by flattening and a softmax activation to produce a probability distribution over the 361 board intersections.
- The **value head** uses global average pooling to aggregate spatial features, followed by a fully connected layer with 128 hidden units and ReLU activation, and a final sigmoid output representing the estimated probability of winning.

Despite the increased architectural sophistication, careful design choices allow the total number of trainable parameters to remain at **98,167**, well below the imposed limit.

3.4.2 Hyperparameters

The main hyperparameters used for this version are summarized below:

- Number of input feature planes: 31
- Board size: 19×19
- Number of convolutional channels: 47
- Number of hybrid blocks: 6
- Kernel size of depthwise convolutions: 7×7
- Hidden units in the value head: 128
- Batch size: 128
- Number of training epochs: 2000

An L_2 regularization coefficient of 10^{-4} is applied to the dense layers in the value head to improve generalization.

3.4.3 Optimization and Loss Functions

The model is trained using the Adam optimizer with a learning rate of 10^{-3}. Training follows a multi-task objective combining policy and value prediction.

Categorical cross-entropy is used as the loss function for the policy head, while mean squared error is used for the value head. The total loss is defined as a weighted sum of the two components, with weights set to 1.0 for the policy loss and 0.4 for the value loss. This configuration maintains a strong emphasis on accurate move prediction while still encouraging reliable position evaluation.

3.4.4 Training Procedure and Evaluation

Training is performed in an online fashion, with new batches of positions generated at each epoch. Data augmentation through random geometric symmetries of the Go board is applied systematically during training to improve robustness and enforce symmetry awareness.

The model is trained for up to 2000 epochs with a mini-batch size of 128. Periodic validation is conducted on a separate validation set without augmentation to monitor performance. Evaluation metrics include total loss, policy accuracy, and mean absolute error of the value prediction.

This fourth version significantly improves the expressive power of the network compared to earlier architectures, while maintaining stability and efficiency under tight parameter constraints.

3.5 Version 5: Optimized Hybrid ConvNeXt with Extended Training

The fifth version represents the final and best-performing architecture among all iterations. It retains the hybrid ConvNeXt-based backbone with 6 stacked HybridBlocks, integrating Local Channel Attention (LCA), Squeeze-and-Excitation (SE), and Global Context (GC) blocks, achieving a compact yet expressive network with **98,167 trainable parameters**.

Compared to Version 4, this architecture is identical in structure, but the training procedure was extended to further exploit its representational capacity and stability.

3.5.1 Network Architecture

The network input is a tensor of shape $(19, 19, 31)$ representing the Go board state. A convolutional stem composed of a 3×3 convolution, batch normalization, and ReLU activation projects the input into a 47-channel representation.

The trunk consists of six stacked *hybrid blocks*, each combining:

- A ConvNeXt block with a depthwise 7×7 convolution followed by a pointwise bottleneck expansion with GELU activation,
- Local Channel Attention (LCA),
- Squeeze-and-Excitation (SE),
- Global Context (GC) block,

all wrapped in a residual connection to ensure stable gradient flow.

The network branches into two heads:

- **Policy head:** 1×1 convolution, flattening, softmax over 361 moves.
- **Value head:** Global average pooling, Dense(128, ReLU, L2 regularization), Dense(1, sigmoid) estimating the probability of winning.

3.5.2 Hyperparameters

The key hyperparameters for this version are summarized below:

- Number of input planes: 31
- Board size: 19×19
- Number of convolutional channels: 47
- Number of hybrid blocks: 6
- Kernel size for depthwise convolution: 7×7
- Hidden units in value head: 128
- Batch size: 128
- Number of training epochs: initially 2000, finally 10000

An L_2 regularization coefficient of 10^{-4} is applied to the dense layers in the value head to improve generalization.

3.5.3 Optimization and Loss Functions

The model is trained with the Adam optimizer ($\text{lr} = 10^{-3}$) and a multi-task loss:

- Policy head: categorical cross-entropy
- Value head: mean squared error (weighted by 0.4 relative to policy loss)

Metrics include policy accuracy and value mean absolute error (MAE). This weighting emphasizes accurate move prediction while still encouraging reliable position evaluation.

3.5.4 Training Procedure and Evaluation

Training proceeds in two stages:

1. **Policy-only pre-training:** 200 epochs to stabilize the backbone. Value head is present but low-priority.
2. **Full training:** policy + value heads jointly trained for 2,000 epochs with batch size 128. Data augmentation through random board symmetries is applied systematically.

Periodic evaluation every 10 epochs measures total loss, policy loss, value loss, policy accuracy, and value MAE. Garbage collection is performed every 5 epochs to reduce memory pressure.

Given that this architecture produced the best results, it was eventually decided to extend training to **10,000 epochs** to fully exploit its capacity.[1]

[1] In a notebook, outputs can be lost during long training runs even with GPU acceleration. Therefore, the same code and hyperparameters were executed in a standalone Python script with 10,000 epochs, ensuring full reproducibility and complete logging. Only the number of epochs differed. This approach avoided potential session disconnections that can occur when training for extended periods.

Model saving and visualization: The trained model is saved as `model.h5`, and training metrics are plotted and stored as PNG files: policy accuracy, policy loss, value loss, value MAE, and total loss.

This extended training ensures the final model achieves maximum performance while retaining the efficient and expressive hybrid ConvNeXt + attention design.

Chapter 4

Results and Discussion

The results presented in this chapter correspond to the final evaluation of each model version on the validation set after the completion of training. The following sections summarize the quantitative results and provide a discussion on their implications.

4.1 Results

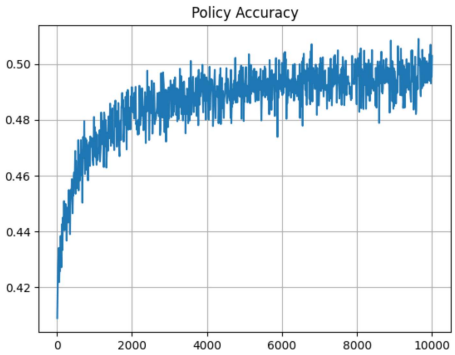
Table 4.1 summarizes the key metrics obtained for all five versions of the Go-playing neural network, including total loss, policy loss, value loss, policy accuracy, and value mean absolute error (MAE).

Version	Total Loss	Policy Loss	Value Loss	Policy Accuracy	Value MAE
1	6.5451	3.1322	0.1204	0.3731	0.2925
2	2.5820	2.4333	0.1160	0.3827	0.2751
3	2.3069	2.1745	0.1203	0.4251	0.2925
4	2.0025	1.9680	0.0835	0.4532	0.2237
5	1.7549	1.7234	0.0714	0.5030	0.2047

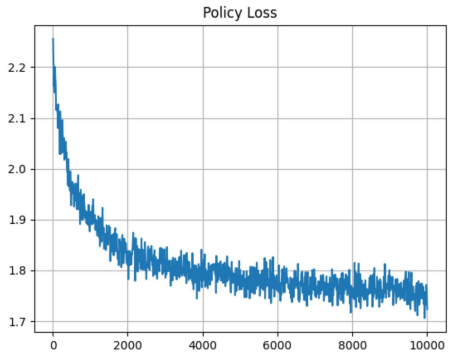
Table 4.1: Summary of validation results for all model versions.

From Table 4.1, it is evident that each successive version improves over the previous one, with Version 5 showing the best performance across all metrics. The total loss decreases consistently from Version 1 to Version 5, while policy accuracy increases above 50%, and value MAE decreases to 0.2047. These results indicate that the hybrid ConvNeXt architecture with attention mechanisms effectively captures both move probabilities and position evaluation.

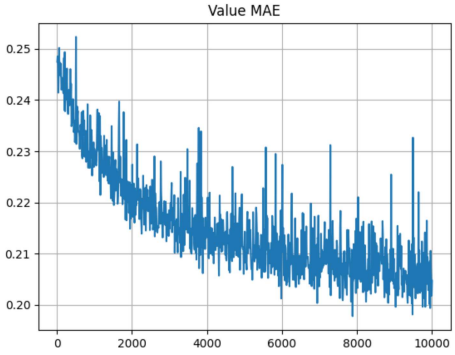
For the final model, additional insight into the training dynamics is provided through the visualization of key metrics. The following plots illustrate the evolution of key metrics for Version 5 over epochs. The x-axis represents epochs, while the y-axis represents the corresponding metric value. Five separate plots are provided for each metric:



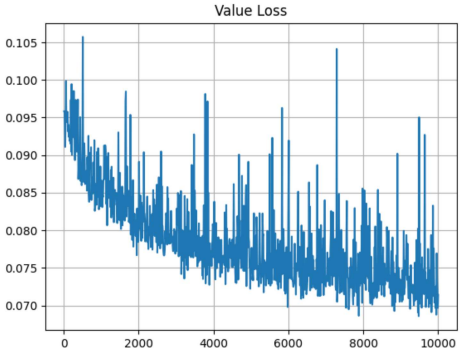
(a) Policy Accuracy over epochs for Version 5.



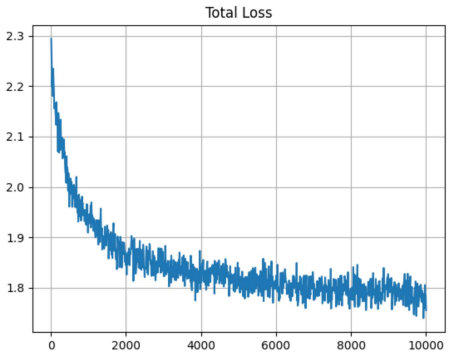
(b) Policy Loss over epochs for Version 5.



(c) Value MAE over epochs for Version 5.



(d) Value Loss over epochs for Version 5.



(e) Total Loss over epochs for Version 5.

4.2 Discussion

The plots demonstrate steady improvements in both policy accuracy and value prediction as training progresses. Notably, even though the final Version 5 training was extended to **10,000 epochs** using a separate Python script, the gains compared to the initial 2,000-epoch

training are relatively modest. For example, policy accuracy improved by roughly 0.02 after an additional 8,000 epochs, and total loss decreased by approximately 0.1 over the same interval.

These observations suggest that the network has approached the limit of its representational capacity under the current architecture and dataset. Further improvements are likely to be incremental unless additional innovations, such as larger datasets, novel regularization strategies, or modifications to the architecture, are introduced.

Moreover, given the imposed constraint of keeping the number of trainable parameters under 100,000, achieving substantially better results is challenging. Compared to state-of-the-art Go models with significantly more parameters, this compact network demonstrates impressive performance, indicating that Version 5 effectively balances expressivity, efficiency, and stability.

Overall, the results indicate that the hybrid ConvNeXt architecture with attention mechanisms is a strong candidate for efficient Go-playing models under strict parameter constraints, and further performance gains within this budget would be relatively difficult to achieve.

Chapter 5

Conclusion

This work focused on the development of compact neural network architectures for the game of Go, under a strict constraint of keeping the total number of trainable parameters under 100,000. Five successive versions of the model were implemented, each progressively improving in policy prediction and value estimation.

Version 1 established the baseline performance, while Versions 2 and 3 introduced deeper convolutional structures and incremental improvements. Version 4 incorporated hybrid ConvNeXt blocks combined with multiple lightweight attention mechanisms (Local Channel Attention, Squeeze-and-Excitation, and Global Context), significantly increasing representational capacity. Version 5 further refined this architecture and included a dedicated policy-only pre-training phase, resulting in the best overall performance with a total loss of 1.7549, policy accuracy of 0.5030, and value MAE of 0.2047.

The final training of Version 5 was extended to 10,000 epochs in a separate Python script to avoid losing output during long notebook executions. Observed gains beyond 2,000 epochs were modest, suggesting the model has reached a near-optimal configuration within the parameter budget. Substantial improvements would likely require a larger model, richer input features, or more advanced architectural innovations.

Overall, this study demonstrates that careful design of hybrid convolutional and attention-based architectures allows efficient Go-playing networks to achieve strong performance under strict parameter limitations. Achieving substantially better results appears challenging under the 100,000 parameter constraint.

Future directions include:

- Exploring larger-scale architectures while maintaining computational efficiency,
- Investigating self-supervised pre-training methods to leverage unlabeled Go game data more effectively,
- Incorporating reinforcement learning with Monte Carlo Tree Search to enhance policy accuracy and strategic performance,
- Studying alternative attention mechanisms or architectural blocks that enhance spatial reasoning without significantly increasing parameter count.

In conclusion, Version 5 represents a robust and efficient architecture for Go-playing AI within strict parameter limitations, balancing accuracy, generalization, and computational efficiency, and providing a strong foundation for potential future enhancements.

References

Kocsis, L. and Szepesvári, C. (2006), Bandit based monte-carlo planning, *in* 'Proceedings of the 17th European Conference on Machine Learning', ECML'06, Springer-Verlag, p. 282–293.

Liu, Z., Mao, H., Wu, C.-Y., Feichtenhofer, C., Darrell, T. and Xie, S. (2022), A convnet for the 2020s, *in* 'Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)', pp. 11976–11986.

Silver, D., Hubert, T., Schrittwieser, J., Antonoglou, I., Lai, M., Guez, A., Lanctot, M., Sifre, L., Kumaran, D., Graepel, T., Lillicrap, T., Simonyan, K. and Hassabis, D. (2018), 'A general reinforcement learning algorithm that masters chess, shogi, and go through self-play', *Science* **362**(6419), 1140–1144.

Wu, D. J. (2020), 'Accelerating self-play learning in go'.